

A Structural Assessment of a Streamlit-Based Storefront Prototype and a Recommended Architecture for Production-Grade Web/Application Delivery

Case Study: the ShopAssist Client UI Module

Technical Report — UI Engineering Track
Capstone Programme in Generative AI — Client Engineering Note TR-UI-01

Abstract

This report examines the client-side implementation of ShopAssist, a merchandise-storefront prototype built as part of a Generative-AI capstone project, with the storefront UI realised in Python using the Streamlit rapid-application framework and a conversational assistant integrated as a floating panel. The prototype's purpose is to demonstrate an end-to-end retrieval-and-agent-orchestration pipeline behind a usable, demo-appropriate interface; it is not, and was never intended to be, a production storefront. This document treats the prototype as a case study in rapid UI prototyping and derives from it a set of structural observations and a recommended architecture for a team that wishes to carry the same product concept, and the same backend contract, into a production-grade delivery using native Web or native application technologies. The assessment covers project structure, execution model, styling approach, state management, and packaging, and closes with a phased migration strategy that preserves the existing backend integration surface throughout.

Keywords: *rapid prototyping, Streamlit, single-page application architecture, conversational UI, design systems, production readiness, migration strategy.*

1. Introduction

1.1 Purpose and Scope

The subject of this assessment is the client module of the ShopAssist system, a self-contained repository responsible for rendering a merchandise catalogue, cart, checkout, and order-history flow, together with an AI shopping assistant reachable from every page. The module communicates with a single backend endpoint over HTTP and is otherwise self-sufficient, drawing its catalogue and account data from bundled fixture files. This report inventories that module's structure, characterises the architectural pattern it follows, and evaluates that pattern's fitness for a production deployment rather than a demonstration one.

The scope is deliberately limited to the client (presentation) tier. The backend's orchestration, retrieval, and guardrail layers are treated as a fixed, versioned contract that any replacement client must continue to honour, and are discussed only insofar as the contract shapes client-side decisions (Section 3.4, Section 6.4).

1.2 Context within the Capstone Program

Capstone teams building conversational-AI products typically allocate the majority of their engineering effort to the reasoning and retrieval pipeline, with the client left as a thin, fast-to-iterate layer. Streamlit is a common choice for this reason: it allows a small team to produce a usable, styled interface in Python without a separate front-end toolchain. That trade-off is appropriate for a time-boxed demonstration and is not, in itself, a defect; the purpose of this report is to make the trade-off explicit and to give the next engineering phase a concrete path away from it, should the product move toward a wider audience or a longer operational lifetime.

2. System Overview

2.1 Functional Summary

- A merchandise catalogue with product detail, cart, and a mocked checkout flow.
- An order-history view and a minimal account/profile page.
- A floating conversational assistant (order tracking, product Q&A, and a support-escalation flow), reachable from any page.
- A gate screen that establishes a stable per-session user identity, used to attribute assistant conversations to a user.

2.2 Technology Stack

Layer	Technology	Notes
UI framework	Streamlit ≥ 1.46	Server-rendered, full-script re-execution per interaction (Section 4.1).
Language	Python 3.12	Single language across UI and (separately) the backend service.
Styling	Hand-written CSS, injected inline	One stylesheet, string-concatenated into the page on every load.
Client–server integration	One JSON/HTTP endpoint	Session/user/text in, agent reply and metadata out (Section 3.4).
Data fixtures	Static JSON files	Catalogue, orders, profile, FAQ — no database in this tier.
Packaging	Docker (single-service compose)	Host-native or container-network backend, selected by environment.

3. Project Structure and Module Inventory

3.1 Directory Layout

```
shopassist-client/
|-- app.py                endpoint, route table, CSS/logo injection
|-- config.py / config.yaml layered settings (YAML defaults, env overrides)
|-- mock_gateway.py       stand-in backend for offline UI development
|-- pages/                one module per route (login, home, catalog, product,
|                           cart, checkout, orders, profile, about)
|-- components/           shared chrome: navbar, footer, product card, cart widget
|-- chatbot/              assistant panel, API client, and voice-input component
|   |-- voice_input/frontend/ hand-rolled JS component (Section 4.2)
|-- data/                 catalogue, orders, profile, FAQ - static JSON fixtures
|-- utils/                logging, session-state helpers, shared constants
|-- styles/styles.css      single project stylesheet (Section 4.3)
|-- assets/               logo, banners, product imagery
`-- Dockerfile, docker-compose.yml container build and standalone run
```

Figure 1. Top-level directory layout of the client module.

3.2 Module Responsibilities

Module	Responsibility
app.py	Process endpoint; holds the route table and dispatches to a page module based on session state; injects the stylesheet and brand logo once per run.
pages/*.py	One render function per storefront screen; reads and writes session state directly; no shared page-level abstraction beyond a common render() signature.
components/*.py	Presentational fragments reused across pages (navigation bar, footer, product card, cart line item).
chatbot/chat_ui.py	Assistant panel: message history, quick-action prompts, the send/clear controls, and a typing-indicator/streaming reveal effect.
chatbot/api_client.py, models.py	The only network boundary in the module; a thin, framework-free HTTP client and response schema kept in lock-step with the backend contract.
chatbot/voice_input.py + frontend/	A speech-to-text mic control, implemented as a custom Streamlit component (Section 4.2).
utils/helpers.py	Session-state bootstrap/restore, logging setup, and chat-history persistence helpers.
config.py, config.yaml	Layered configuration: repository-committed defaults overridden by environment variables at deploy time.

3.3 Size Distribution and Complexity Indicators

The module comprises approximately 2,300 lines of Python across twenty non-empty source files, a single 628-line stylesheet, and one 132-line hand-written HTML/JavaScript component. No automated test suite (unit or end-to-end) is present in the repository at the time of writing; verification during development has relied on manual and ad-hoc browser-driven checks. Table 2 lists the largest modules by line count.

Module	Lines	Observation
chatbot/chat_ui.py	436	Largest module in the tree; combines layout, seven distinct widget callbacks, streaming-reveal animation, and inline JavaScript in one file — a candidate for decomposition regardless of framework (Section 5).
utils/helpers.py	288	Session bootstrap, logging, and persistence utilities collected in one module.
mock_gateway.py	240	Offline stand-in for the backend; not part of the deployed client, retained for local development.
pages/home.py	139	Largest single page module; banner carousel and promotional rail.
styles/styles.css	628	One monolithic, ungrouped stylesheet covering every page and the assistant panel (Section 4.3).

4. Architectural Characteristics of the Prototype

4.1 Execution and Rendering Model

Streamlit executes the entire Python script top to bottom on every user interaction; there is no persistent client-side application state or virtual-DOM-style partial update, and no URL-addressable client-side router — navigation is implemented as a string key in server-held session state, dispatched through a dictionary of render functions in `app.py`. This is an efficient authoring model for a small team (a widget's callback and its resulting view are colocated in one function), but it does not produce deep-linkable pages, browser back/forward semantics, or the fine-grained, component-level reactivity that a production single-page application framework provides.

4.2 Client-Side Interactivity via Hand-Rolled Components

Because the framework has no built-in speech-input widget, voice capture is implemented as a custom component: a small, dependency-free HTML/JavaScript page loaded into a sandboxed iframe, communicating with the Python process through the framework's `postMessage`-based protocol. The component itself calls the browser's native Web Speech API directly and is, in isolation, simple; the surrounding plumbing — a handshake message, a frame-height negotiation, and a manual duplicate-suppression guard on the Python side — exists solely to bridge that native browser capability back into the server-executed script model described above. A native Web or application client would call the same browser or platform speech API directly from its own event loop, with no bridging protocol required (Section 6.5).

4.3 Styling Approach

Presentation is governed by one repository-wide stylesheet, read from disk and injected as a single inline style block on every page load, together with a small number of CSS attribute-selectors keyed to widgets' explicit identifiers. This is workable at the module's current size but does not scope styles to components, does not benefit from a separate cacheable stylesheet resource, and depends on selector conventions that are internal implementation details of the UI framework rather than a stable public contract.

4.4 State, Routing, and Persistence

Session state is held in the framework's server-side process memory, keyed by a browser session; a single query parameter mirrors the signed-in user identifier so that a page refresh can restore a session. Catalogue, order, and profile data are static JSON fixtures bundled with the client rather than served by a backing data service. This is appropriate for a single-instance demonstration deployment and does not, as it stands, support multiple concurrent application instances sharing one session store, nor a real product/order data service.

4.5 Packaging and Deployment

The module is packaged as a single Docker service with a health check, a non-root runtime user, and an environment-driven switch between a host-native and a container-network backend address. This packaging discipline is already consistent with production practice and is a part of the current implementation this report recommends retaining unchanged through any client rewrite (Section 6.9).

4.6 Authentication Posture

The sign-in screen validates a demonstration rule (the submitted password must equal the submitted user identifier) chosen specifically to exercise both a successful and an unsuccessful sign-in path without standing up a credentials store. The project's own roadmap already lists real alumni single-sign-on as outstanding work; this report treats that item as confirmed, pre-existing scope for the production track (Section 6.6) rather than a newly discovered gap.

5. Suitability Assessment: Rapid Prototyping versus Production Delivery

Table 3 summarises, dimension by dimension, where the current implementation sits and what a production-grade equivalent would require. The comparison is offered in the spirit of a capability gap analysis rather than a critique: every item on the right-hand side is additional engineering scope appropriate to a wider release, not a correction of an error in the prototype.

Dimension	Prototype (current)	Production target
Rendering model	Full server-side script re-execution per interaction	Component-level client-side reactivity; server rendering used for first paint/SEO only
Routing	In-memory route key, no URL	Client-side router with deep-linkable, shareable URLs
Styling	One inline stylesheet, framework-internal selectors	Scoped component styles from a shared design-token system
State management	Server-held session state, single process	Client cache for server data plus a small local UI-state store, horizontally scalable backend
Conversational transport	Single blocking request/response call	Streamed responses over Server-Sent Events or WebSocket
Speech input	Bridged into a native browser API via a custom component protocol	Direct native platform/browser API call

Dimension	Prototype (current)	Production target
Authentication	Demonstration rule (password equals user identifier)	Federated sign-on (e.g. institutional identity provider) with secure session/token handling
Data layer	Static bundled JSON fixtures	A real catalogue/order service with its own persistence tier
Automated testing	None in-repository	Unit, component, and end-to-end test suites gating every change
Packaging	Single Docker service, health-checked	Unchanged — already fit for production (Section 4.5)

6. Recommended Architecture for Production-Grade Delivery

The recommendations below assume the existing backend contract — a single conversational endpoint exchanging a session identifier, a user identifier, and free text, described further in Section 3.4's referenced contract — is retained unchanged, so that client and backend work can proceed independently.

6.1 Presentation Layer

A component-based, statically typed front-end framework (for example React or Vue, written in TypeScript) is recommended for the Web client, rendered with a hybrid strategy: server-side or static generation for the storefront's browsable pages (catalogue, product detail) for first-paint speed and search-engine visibility, and client-side rendering for stateful views (cart, checkout, the assistant panel). Should a native mobile client also be required, a cross-platform toolkit (React Native or Flutter) sharing the same design tokens and API contract is the natural extension of the same architecture.

6.2 State Management and Data Fetching

Server-originated data (catalogue, cart, order history, assistant conversation) is best modelled as cached, revalidated queries (for example via TanStack Query or an equivalent data-fetching layer) rather than hand-rolled request/response plumbing; purely client-local UI state (panel expanded/collapsed, active route) can use a minimal store (Zustand, Redux Toolkit, or framework-native state) kept deliberately small.

6.3 Styling and Design System

Component-scoped styling (CSS Modules, or a utility-first system such as Tailwind CSS) built against a shared token set — the palette, spacing, and typography already defined in this project's configuration translate directly into such tokens — replaces the single global stylesheet, enabling per-component caching, no accidental cross-page selector collisions, and reuse across a Web and native client.

6.4 Real-Time Conversational User Experience

The assistant's replies should stream to the client incrementally over Server-Sent Events or a WebSocket connection as the backend agent produces them, rather than the client waiting on one blocking call and animating a reveal after the fact. This removes the artificial typing-indicator/reveal choreography entirely and improves perceived latency, which matters proportionally more once the backend's per-turn processing time is outside the

client's control.

6.5 Native Browser and Platform Capabilities

Speech-to-text, and any comparable native capability (clipboard access, push notifications, on-device biometrics for a mobile client), should be called directly from the client's own code rather than bridged through an intermediate protocol designed to work around a server-rendered framework's limitations. This both removes an entire integration surface and gives direct access to browser features — interim transcripts, confidence scores, alternative recognition results — that a bridging layer would otherwise have to re-implement one at a time.

6.6 Authentication and Session Management

The demonstration credential rule should be replaced by a federated sign-on flow appropriate to the target user population (for example an institutional identity provider for an alumni audience), issuing short-lived, securely stored session tokens rather than a client-held plain user identifier.

6.7 Testing Strategy

A production client should carry unit and component tests (for example Jest with a component testing library) alongside an end-to-end browser suite (Playwright or Cypress) exercising the same golden paths and edge cases identified during this assessment's own manual verification, run automatically on every change rather than ad hoc.

6.8 Observability and Quality Gates

Structured client-side logging, front-end error tracking, and real-user performance monitoring (Core Web Vitals or equivalent) give the operating team visibility that a demonstration deployment does not need but a production one does, particularly once the client is decoupled from a single engineer's direct observation of it.

6.9 Deployment and Packaging

The existing containerisation discipline — a single, health-checked Docker service with an environment-driven backend address — should be retained as-is. For a Web client specifically, static build artefacts are additionally well suited to a content-delivery network in front of the container, with the container itself serving only the dynamic, streamed portions of the application.

7. Migration Strategy

A full rewrite is not the only, or necessarily the best, path from the current prototype to the architecture described in Section 6. Because the backend contract is already an independent, versioned boundary, a phased strangler-pattern migration — standing up the new client alongside the existing one and moving one route at a time — lets both implementations serve traffic concurrently during the transition. Table 4 sketches one reasonable phasing; the exact grouping is a project-management decision outside this report's scope.

Phase	Scope	Rationale
1	Design-token extraction and a shared component-style guide	Decouples visual identity from any one framework before any page is rewritten.

Phase	Scope	Rationale
2	Storefront browsing routes (catalogue, product detail)	Highest traffic, lowest state complexity, best early return on server-rendering/SEO investment.
3	Cart, checkout, order history	Stateful flows; benefits most from a proper client-side data-fetching layer.
4	Assistant panel with streaming transport and native speech input	Highest-complexity surface; migrated last, once the surrounding shell is stable.
5	Authentication cut-over to federated sign-on	Coordinated with identity-provider availability; can proceed independently of the UI phases above.

8. Conclusion

The ShopAssist client fulfils its stated purpose: it demonstrates a full conversational-commerce flow atop a real orchestration backend, built and iterated on quickly by a small team. Every gap identified in this report is a known and expected consequence of that trade-off rather than an implementation defect, and each has a well-established production pattern to replace it — a component-based front-end framework, a proper client-side data and state layer, streamed conversational transport, native platform APIs called directly, and a real authentication and data tier. Because the backend integration contract is already a stable, independent boundary, these changes can be adopted incrementally, behind the same API, without requiring the orchestration layer to change in step.

9. References

- World Wide Web Consortium / WHATWG. *Web Speech API*.
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*.
<https://www.w3.org/WAI/WCAG22/quickref/>
- OWASP Foundation. *Application Security Verification Standard*.
<https://owasp.org/www-project-application-security-verification-standard/>
- React core team. *React documentation*. <https://react.dev>
- Vercel. *Next.js documentation*. <https://nextjs.org/docs>
- WHATWG. *Server-Sent Events*, part of the HTML Living Standard.
<https://html.spec.whatwg.org/multipage/server-sent-events.html>
- TanStack. *TanStack Query documentation*. <https://tanstack.com/query>
- Streamlit Inc. *Custom components*. <https://docs.streamlit.io/develop/concepts/custom-components/intro>

10. Appendix A: Full Source File Inventory

File	Lines	Category
app.py	106	Entrypoint / routing
config.py	69	Configuration

File	Lines	Category
mock_gateway.py	240	Local dev tooling
pages/login.py	57	Page
pages/home.py	139	Page
pages/catalog.py	56	Page
pages/product.py	96	Page
pages/cart.py	67	Page
pages/checkout.py	119	Page
pages/orders.py	80	Page
pages/profile.py	82	Page
pages/about.py	56	Page
components/navbar.py	58	Component
components/footer.py	19	Component
components/product_card.py	67	Component
components/cart_widget.py	29	Component
chatbot/chat_ui.py	436	Assistant UI
chatbot/api_client.py	95	Assistant / API
chatbot/models.py	52	Assistant / API
chatbot/voice_input.py	44	Assistant / voice component
chatbot/voice_input/frontend/index.html	132	Assistant / voice component (JS)
utils/helpers.py	288	Utilities
utils/constants.py	40	Utilities
styles/styles.css	628	Styling

Four additional `__init__.py` package markers (zero lines each) are omitted from the table above. Data fixtures (five JSON files under `data/`) and static assets under `assets/` are likewise omitted as non-source content.